Um Modelo para Gerenciamento de Perfis de Entidades Através de Inferência em Trilhas

André Wagner
Jorge Luis Victória Barbosa
Débora Nice Ferrari Barbosa

1 Introdução

Atualmente, as aplicações têm se concentrado na coleta de informações a respeito dos usuários, com o objetivo de conhecer suas preferências, possibilitando a oferta de serviços personalizados. Assim, estas aplicações organizam seu conhecimento a respeito do usuário em perfis de usuário, os quais armazenam as informações em um formato baseado em um modelo de usuário específico.[1]

Sistemas sensíveis a contexto são aplicações nas quais o contexto atual de um usuário é levado em consideração na disponibilização de serviços. Dey[2] define contexto como qualquer informação que pode ser utilizada para caracterizar a situação de uma entidade, onde uma entidade pode ser uma pessoa, local ou objeto considerado relevante à interação entre o usuário e a aplicação. Desta forma, o perfil torna-se parte essencial na descoberta de informações pertinentes aos usuários que possam ser utilizadas na construção de um contexto.

Os ambientes de computação ubíqua são parte de um cenário no qual o custo de dispositivos é baixo e a capacidade e disponibilidade são altos, sendo possível prover serviços e informações de modo amplo e contínuo[3]. Esta visão vem sendo potencializada pela diminuição do custo e aumento da capacidade de memória e processamento dos dispositivos computacionais, bem como pela cada vez maior disponibilidade de recursos de localização e conectividade. A computação ubíqua enfatiza também a "tecnologia calma", onde a sobrecarga de informação é substituída pela disponibilização de recursos de forma inteligente, tomando a atenção do usuário apenas nos momentos em que é mais relevante.[4]

A capacidade de uma aplicação inferir o contexto de uma entidade pode ser amplificada através do uso de trilhas[5]. Uma trilha é o histórico de contextos visitados por uma entidade, e das ações realizadas pela entidade nos contextos.

Existem trabalhos que utilizam trilhas como forma de armazenar e processar informações de um usuário[6, 5]. No entanto, nestes trabalhos, o modelo do usuário já está pré-definido, e a trilha é utilizada apenas para alimentar informações específicas do perfil.

Este trabalho busca aprofundar esta abordagem apresentando um modelo para gerenciamento de perfis através de inferência em trilhas. O objetivo do modelo é permitir que qualquer aplicação que faça uso de trilhas possa ser capaz de montar seu próprio modelo de usuário e definir as regras necessárias para que este perfil seja inferido a partir das trilhas.

A Seção 2 apresenta conceitos gerais a respeito de gerenciamento de perfis, e na Seção 3 são comparados trabalhos relacionados. A Seção 4 apresenta o modelo, e a Seção ?? conclui comparando o modelo apresentado com os relacionados, e traçando possíveis trabalhos futuros.

2 Conceitos Gerais

A coleta de informações de um usuário pode ser explícita ou implícita[1]. Na coleta explícita, é requisitado ao usuário que preencha informações através de um formulário pré-definido, enquanto na coleta implícita, a informação é inferida a partir do estudo do comportamento do usuário.

Informações a respeito de um usuário colhidas por uma aplicação podem ser úteis para outras aplicações que tenham o mesmo usuário [1]. No en-

tanto, os diferentes modelos de usuário utilizados pelas aplicações tornam difícil a utilização de informações de um mesmo perfil de usuário por várias aplicações. Além disso, algumas das informações do usuário podem ser válidas apenas dentro de um contexto específico, o que dificulta ainda mais que diferentes aplicações compartilharem o mesmo perfil.

Visando resolver estas questões, o conceito de personalização inter-sistemas foi introduzido [7] como forma de compartilhar informações de usuários entre diferentes aplicações. Como o maior problema deste tipo de modelo é o alinhamento entre as diferentes abordagens de personalização, o estudo de sistemas de modelos de usuário é atualmente o principal desafio da pesquisa na área de personalização.[1]

O IEEE Standard Computer Dictionary [8] define interoperabilidade como "a capacidade de dois ou mais sistemas ou componentes trocarem informações e utilizar a informação que foi trocada." No âmbito da personalização, interoperabilidade é a capacidade de acessar e interpretar informação derivada de múltiplas fontes heterogênias e integrar esta informação em um modelo de usuário comum[9]. Os dois principais modelos de interoperabilidade são: (i) interoperabilidade sintática, onde as diferenças entre os modelos de usuário são resolvidos em nível de aplicação, e (ii) interoperabilidade semântica, onde as diferenças entre os modelos são resolvidos em nível de conhecimento.

As duas principais abordagens utilizadas em sistemas de modelos de usuários interoperáveis são [1]:

Baseados em padrões: são baseados numa visão *top-down*, definindo um padrão que todas as aplicações envolvidas devem observar;

Baseados em mediação: se baseiam numa visão *bottom-up*, onde ocorre algum tipo de reconciliação entre os diferentes modelos de representação.

Descrever conceitos de perfis relacionados a ontologias.

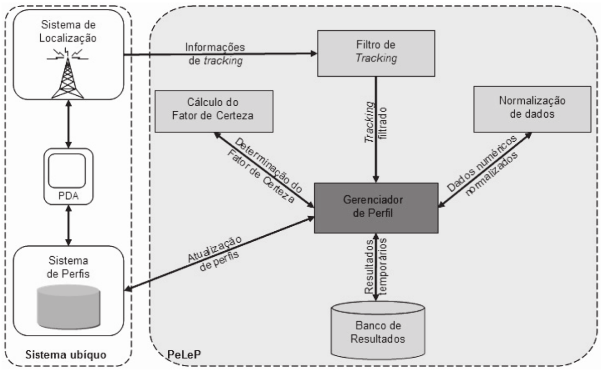

Figura 1: Arquitetura do PeLeP[10]

3 Trabalhos relacionados

Esta seção apresenta trabalhos relacionados a com o modelo proposto, bem como uma comparação entre os mesmos.

3.1 PeLeP

O projeto PeLeP, apresentado por Levis et al[10], integra a utilização de perfis aperfeiçoados automaticamente com ambientes de educação ubíqua.

Barbosa et al compreendem *educação ubíqua* como sendo processos educacionais que ocorrem em qualquer lugar, a qualquer tempo e com qualquer dispositivo, de forma contínua, contextualizada e integrada ao cotidiano do aprendiz. Desta forma, o conhecimento presente no dia-a-dia das mais diferentes formas e em diferentes locais podem ser relacionados com processos educacionais direcionados aos aprendizes [11].

O PeLeP, cujo modelo é ilustrado na Figura 1, é dedicado ao gerenciamento de perfis de aprendizes em um ambiente ubíquo de ensino e aprendizagem. O modelo administra os perfis automaticamente, inferindo informações através do histórico do aprendiz. As alterações realizadas no perfil refletem o comportamento do aprendiz no ambiente ubíquo, levando em consideração o seu modelo de mobilidade e de contexto.

No PeLeP, o *perfil do aprendiz* consiste em um perfil contendo campos pré-definidos, voltados para o ambiente de educação ubíqua. Os campos são distribuídos nas seguintes categorias: identificação, objetivos, agenda, segurança, relacionamen-

tos, competências e preferências.

É realizado um acompanhamento (*tracking*) do histórico do aprendiz, onde são armazenados os contextos visitados pelo usuário, bem como os conteúdos, dispositivos e aplicativos utilizados em cada contexto. Os valores relativos das variáveis numéricas recebidas no *tracking* são regularizados através de um processo de normalização de amplitude. Como diferentes valores podem ter diferentes impactos para a determinação da preferência de um usuário, calcula-se o Fator de Certeza (FC), de acordo com a equação 1, onde f_i é o valor normalizado para o fator, p_i o seu percentual, e n o número de fatores considerados.

$$FC = \frac{\frac{\sum_{i=0}^{n} f_i p_i}{100}}{n} \quad (1)$$

O fator tempo também é levado em consideração, onde Fatores de Certeza mais antigos tem um impacto menor. Assim, por exemplo, se o usuário trocar o seu aplicativo preferido, à medida que o tempo for passando o impacto do aplicativo anterior será cada vez menor.

O processo de atualização do modelo do aprendiz é então realizado através de regras pré-definidas, que levam em conta as ocorrências e o seu fator de certeza. As regras suportam conclusões a partir de evidências antecedentes, e cada categoria de informação possui regras diferentes.

O PeLeP foi validado através da integração com o Local (*Location and Context-Aware Learning*), um sistema de educação ubíqua sensível a contexto [12].

3.2 Chellouche et al

O artigo de Chellouche et al[13] discute as dificuldades atuais no desenvolvimento de aplicativos para dispositivos móveis, uma vez que os mesmos possuem uma grande variedade de resolução, memória, poder computacional, entre outros. Além disso, a qualidade da conexão de um usuário pode variar significativamente, e isto também deve ser levado em consideração no desenvolvimento.

O conjunto de informações que caracteriza um usuário é chamada de *perfil de usuário*. Com a intenção de compartilhar o contexto do usuário entre serviços e minimizar as mudanças no desenvolvimento de aplicações sensíveis a contexto, os autores propõem como solução um *middleware* que gerencia o perfil de um usuário dinamicamente e supre às aplicações todas as informações que elas podem precisar durante o período de adaptação.

O perfil do usuário é constituído por seu contexto, onde a definição de contexto utilizada é aquela dada por Dey[2], na qual contexto refere-se a qualquer informação que pode ser utilizada para caracterizar a situação de uma entidade. Assim, o perfil de usuário é dividido em cinco componentes:

Perfil geral: refere-se às informações básicas, como nome, endereço, telefone, idade, sexo, entre outros;

Perfil do dispositivo: contém informações a respeito dos dispositivos do usuário, tais como marca, capacidade, sistema operacional, entre outros;

Perfil de rede: descreve as redes às quais o usuário pode se conectar, com suas características;

Perfil dos serviços: registra as informações a respeito de serviços, como nome, versão, porta e conteúdo;

Perfil de contexto: contém informações a respeito do ambiente do usuário, como hora, data e localização, aplicações em execução e banda disponível. É considerada a parte dinâmica do perfil, composta por dados voláteis.

O *middleware* é apresentado segundo ilustrado na Figura 2. O *middleware* é composto dos seguintes módulos:

Gerenciador de perfil de usuário: módulo central da arquitetura, que interage com todos os outros módulos com o objetivo de construir o perfil correto;

Gerenciador de contexto: monitora o ambiente, coletando informações com o objetivo de atualizar a parte dinâmica do perfil;

Gerenciador de consultas: módulo responsável por validar e salvar um modelo de consulta para cada aplicação;

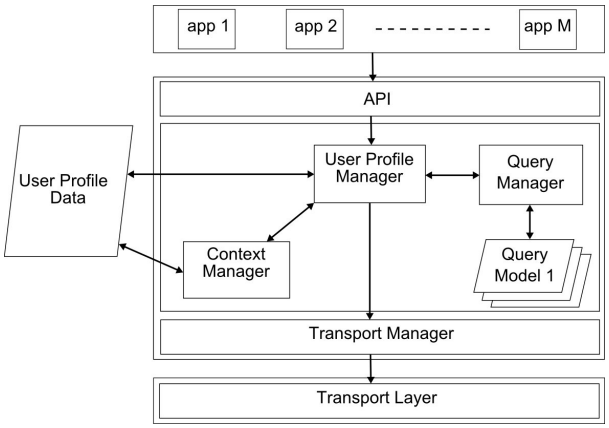

Figura 2: Arquitetura do *middleware* de Chellouche et al[13]

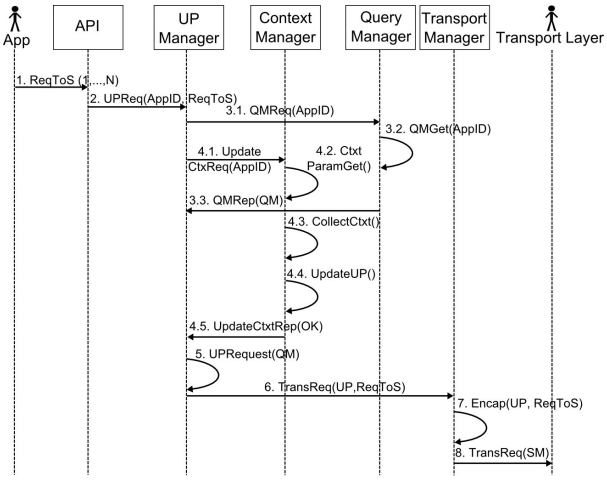

Figura 3: Construção de perfil[13]

Gerenciador de trasporte: faz a comunicação do perfil de usuário para a aplicação;

API : constitui o ponto de interface entre as aplicações e o *middleware*.

O perfil é construído de acordo com a Figura 3. A aplicação solicita os dados ao gerenciador de perfis, através da API, que por sua vez repassa a solicitação ao gerenciador de consultas. Ao mesmo tempo, o gerenciador de contextos é informado que a aplicação está em execução e passa a atualizar o gerenciador de perfis com as informações atualizadas. O perfil de usuário é criado e enviado ao gerenciador de transporte, que retorna o perfil solicitado à aplicação solicitante.

3.3 UUCM

O artigo apresentado por Niederée et al[7] busca responder três indagações, que são apresentadas como sendo os três desafios básicos da personalização:

- Como modelar o usuário e sua situação de modo extensível e unificado, de forma que este modelo possa ser interpretado e usado para personalização por diferentes sistemas?
- Como coletar dados para preencher e atualizar perfis de usuário a partir das interações do usuário com diferentes sistemas baseados em um modelo de contexto de usuário unificado?
- Como habilitar conceitualmente e tecnicamente diferentes sistemas a fazer uso dos perfis de usuário coletados para aperfeiçoamento do suporte ao usuário?

Os autores apresentam como resposta a estas indagações um modelo denominado UUCM, que tem como objetivo modelar características do usuário e de seu contexto. O conceito de contexto utilizado é aquele apresentado por Berenetti et al[14], na qual o contexto pode ser pensado como as informações extras, geralmente implícitas (como associações, fatos, suposições), que podem ser utilizadas para compreender plenamente uma interação, comunicação ou representação de conhecimento.

A abordagem para a modelagem unificada de contextos de usuário é proposta em dois níveis. No nível *abstrato*, são definidos os blocos básicos de construção: contexto de usuário, facetas do modelo de usuário, propriedades principais e dimensões do modelo de usuário. O termo faceta representa as diferentes características do usuário. Este nível define um metamodelo para as dimensões e facetas concretas utilizadas na descrição do modelo de contexto de usuário. Para que a abordagem de personalização multi-sistema seja válida, é necessário que este metamodelo de contexto de usuário seja publicado como uma ontologia compartilhada, e que todos os participantes utilizem este modelo.

No nível *concreto*, um conjunto de dimensões e facetas são definidos. As facetas e dimensões são

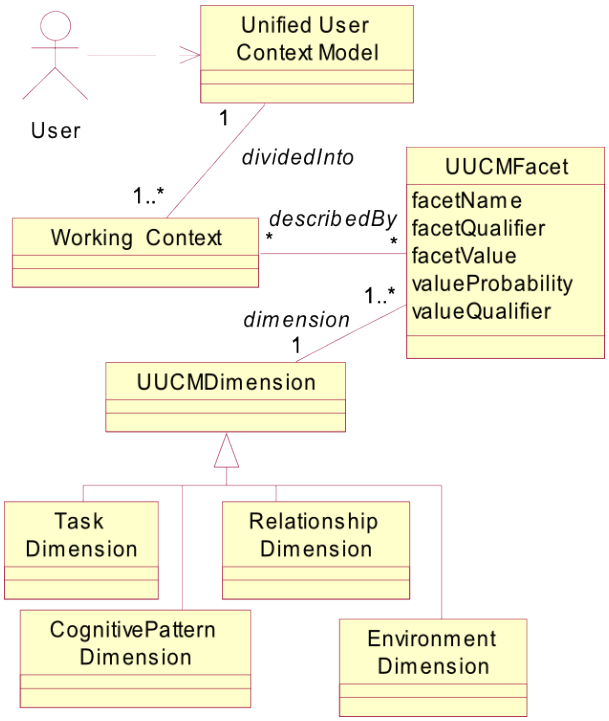

Figura 4: Modelo do UUCM[7]

descritas como parte de uma ontologia adicional que é compartilhada pelos componentes envolvidos na personalização multi-sistemas.

O modelo do UUCM é apresentado na Figura 4. Segundo o modelo, um perfil de usuário (*Unified User Context Model*) é dividido em um conjunto de contextos de trabalho (*Working Contexts*), onde cada contexto é descrito por um conjunto de facetas (*UUCM facet*). Cada faceta é vinculada a uma dimensão (*UUCMDimension*).

O UUCM apenas define o método através do qual o perfil de contexto de usuário é descrito e estruturado para personalização multi-sistema. Para a descrição de perfis de contexto concretos, o UUCM utiliza ontologias (ou vocabulários) para as dimensões, facetas e valores de faceta.

As seguintes dimensões são utilizadas:

Cognitiva (*CognitivePattern*): descreve as dimensões cognitivas do usuário, tais como *áreas de interesse*, *competências* e *preferências*;

Tarefa (*Task*): descreve os objetivos do usuário quando ele interage com sistemas. Utiliza dados tais como *tarefa atual*, *papel do usuário na tarefa* e *histórico da tarefa*;

Relacionamentos (*Relationship*): armazena as entidades com as quais o usuário tem alguma relação;

Ambiente (*Environment*): parâmetros que são tipicamente utilizados para abordagens sensíveis a contexto, tais como *tempo*, *localização*, *dispositivo* e *linguagem*.

O perfil de usuário do UUCM é codificado como um *RDF Schema*[15], ampliado com expressões *OWL*.

O modelo inclui ainda o conceito de *passaporte de contexto*. O passaporte de contexto é uma representação compacta do modelo de contexto atual do usuário para a personalização multi-sistemas. Ele contém informações ontologicamente arranjadas a respeito dos padrões cognitivos (habilidades, interesses, etc), o ambiente (data, local, dispositivo usado) e sua rede pessoal de pessoas e relacionamentos envolvidos.

Para utilizar o passaporte de contexto em um ambiente de personalização multi-sistemas, o usuário "apresenta" o passaporte ao sistema que o usuário deseja utilizar. Como o passaporte está vinculado a uma ontologia compartilhada, é possível que o sistema consiga interpretar parcialmente o passaporte através de uma arquitetura mediadora. Como resultado, dois fluxos são possíveis: (i) do passaporte para o sistema, auxiliando o sistema a compreender o contexto do usuário, e (ii) do sistema para o passaporte, realizando alterações nas informações de contexto do usuário.

3.4 Avaliação dos Ambientes Pesquisados

O PeLeP[10] é um modelo que suporta o aperfeiçoamento automático do perfil do aprendiz em ambientes de educação ubíqua. Ele usa o histórico dos aprendizes nos contextos de um ambiente ubíquo para atualização dos perfis. O perfil do aprendiz no modelo é refinado e enriquecido através de inferências. As inferências são baseadas na mobilidade do aprendiz e no monitoramento das tarefas que ele executa nos contextos de um ambiente de educação ubíqua.

O modelo apresentado por Chellouche et al[13] apresenta um *middleware* voltado para aplicações móveis que pode ser usado por diferentes aplicativos como um servidor central de perfis. Este modelo propõe como solução um *middleware* que gerencia o perfil de um usuário dinamicamente e supre às aplicações todas as informações que elas podem precisar durante o período de adaptação.

O UUCM[7] é um modelo que pode ser utilizado para modelar as características de um usuário e seu contexto em ambientes de personalização de múltiplos sistemas. A proposta diz respeito a uma ontologia básica, baseada em contexto, a qual os sistemas podem estender, desde que os sistemas que usem este modelo concordem em uma ontologia básica.

Embora os modelos apresentados atualizem parte do perfil como forma de oferecer suporte a sensibilidade a contexto, nenhum deles permite que os aplicativos criem novos campos nem definam as regras para atualização dos campos existentes.

A Tabela 1 exibe um comparativo dos trabalhos apresentados, onde os seguintes fatores são levados em consideração:

Descrever porque estes aspectos estão sendo avaliados.

Interoperabilidade: considera se o modelo apresenta interoperabilidade – caso positivo, se a mesma é sintática ou semântica;

Perfis dinâmicos: identifica se o modelo de perfil permite que os perfis sejam atualizados de acordo com eventos externos, sem a interferência do usuário;

Armazenamento: define onde ficam armazenados os perfis do usuário;

Sensível a contexto: considera se o contexto do usuário é levado em consideração na formação do perfil;

Utiliza trilhas: define se as trilhas do usuário são levadas em consideração na construção do perfil;

Domínio: identifica se o perfil é voltado para um domínio específico, ou se o mesmo permite que vários domínios possam ser utilizados.

4 eProfile

O eProfile é um gerenciador de perfis voltado para ambientes distribuídos, que infere dados de perfil através de inferências nas trilhas de uma entidade. A trilha é disponibilizada por um gerenciador de trilhas que se conecta ao eProfile, onde é feita uma mediação entre o formato de dados da trilha e do eProfile.

A figura tradicional do usuário na personalização é estendida pelo conceito de *entidade*. Uma entidade é um conceito, não representando apenas pessoas mas também objetos que possam criar uma trilha e ter um perfil, como um veículo ou um dispositivo móvel. Uma entidade pode ser (i) simples, mapeando os dados de uma única entidade física, ou (ii) composta, onde refere-se a um grupo de entidades. Um exemplo de uma entidade composta seria uma frota de caminhões, onde cada caminhão é uma entidade simples que tem seu perfil único, e onde a frota tem um perfil também.

A ontologia de perfil do eProfile pode ser estendida pelas aplicações, de modo que as aplicações possam armazenar dados de perfil de acordo com um modelo de usuário próprio. Assim, as aplicações podem disponibilizar suas próprias regras para a inferência das informações relacionadas à sua própria estrutura de dados. Os dados inferidos ficam então disponíveis para serem consultados pelas entidades.

Como o eProfile é um modelo que infere informações de forma autônoma a respeito de dados armazenados em uma ontologia, a tecnologia de agentes autônomos foi escolhida para o desenvolvimento.

4.1 Modelo

A Figura 5 apresenta o modelo do eProfile.

As **aplicações** são utilizados pelas entidades para se conectar ao gerencidador de trilhas e ao gerenciador de perfil. Neste modelo, a aplicação se conecta a um gerenciador de trilhas, responsável pelo armazenamento da trilha da entidade. O perfil da entidade será inferido a partir das trilhas registradas no gerenciador.

As trilhas das entidades são buscadas do gerenciador de trilhas pelo eProfile. Como o eProfile pode se conectar a diferentes gerenciadores de trilha (Figura 6), onde em cada um deles a trilha

Trabalho	Interope-rabilidade	Perfis dinâmicos	Armaze-namento	Sensibilidade a contexto	Utiliza trilhas	Domínio
PeLeP [10]	Não	Sim	Servidor	Sim	Parcial	Educação
Chellouche et al [13]	Sintática	Sim	Servidor	Sim	Não	Serviços multimídia
UUCM [7]	Sintática	Sim	Cliente	Sim	Não	Genérico

Tabela 1: Comparação entre os trabalhos apresentados

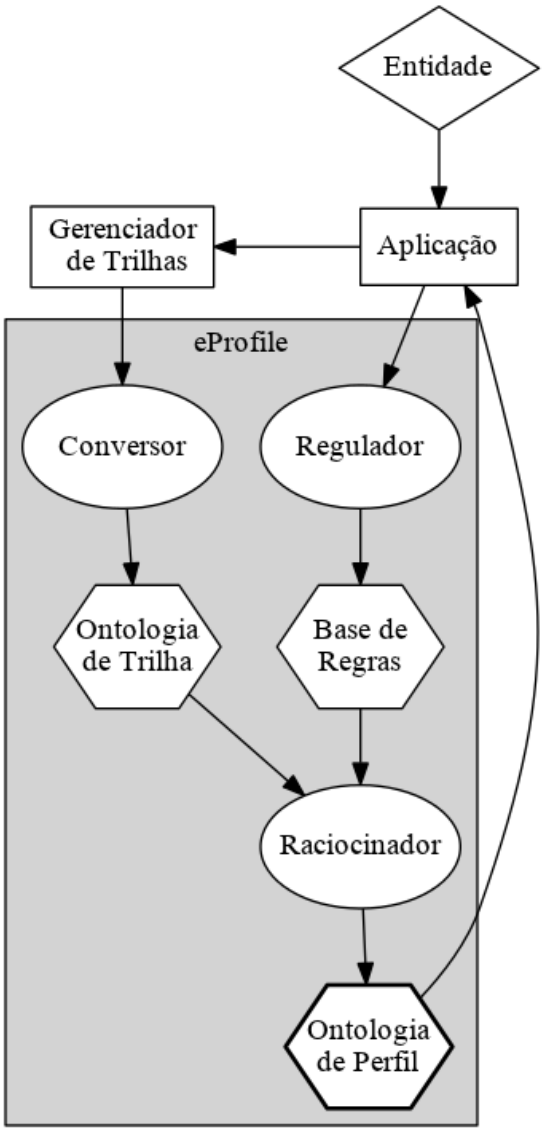

Figura 5: Modelo do eProfile, com as setas representando o fluxo de dados

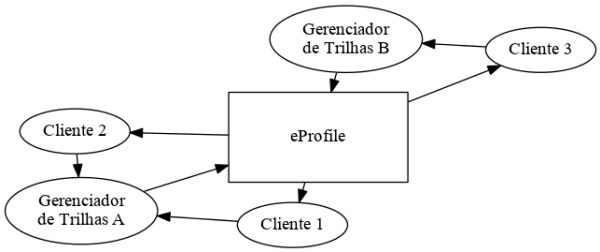

Figura 6: eProfile conectado a mais de um gerenciador de trilhas

tem um formato específico, é necessário que seja realizada uma conversão entre o formato original e o formato utilizado pelo eProfile. Este trabalho é realizado por um agente entitulado **conversor**, que é capaz de realizar a conversão para os formatos de trilha suportados pelo eProfile.

Desta forma, o eProfile não tem a necessidade de conhecer a aplicação – mas precisa conhecer o formato utilizado pelo gerenciador de trilhas.

A trilha do usuário é então armazenada no eProfile em uma **ontologia de trilha**. Esta ontologia possui as classes básicas para a construção de uma trilha, sendo que a mesma pode ser estendida de acordo com a especificação do gerenciador de trilhas que está sendo utilizado. A forma como ocorre esta extensão é detalhada na Seção 4.2.

Cada aplicação tem a possibilidade de montar seu próprio modelo de usuário. Para que o eProfile processe os perfis baseados em um modelo de usuário, ele precisa de três informações: (i) a trilha da entidade, (ii) o modelo de usuário utilizado pela aplicação e (iii) as regras de inferência para o perfil. A trilha da entidade é carregada pelo eProfile a partir do gerenciador de trilhas, enquanto as outras duas informações são enviadas diretamente da aplicação para o eProfile. Desta forma, as regras de inferência definidas pela aplicação serão utilizadas para inferir seu próprio perfil.

O agente responsável pela recepção das regras é o **regulador**. O regulador tem a função de receber as regras, validá-las e inserí-las na **base de regras**. Juntamente com as regras, é especificado a criticidade da informação. O sistema utilizará esta informação para determinar com que freqüência a inferência desta informação será executada.

Uma vez de posse das trilhas, do modelo e das regras, o agente **raciocinador** realiza a inferência na trilha, gerando assim a **ontologia de perfil**. Esta ontologia, da mesma forma que a ontologia de trilha, contém classes básicas a partir das quais as aplicações vão especializar, utilizando-as para armazenar informações próprias. A aplicação pode então conectar-se ao eProfile e buscar o perfil de sua entidade. A Figura 5 exemplifica isso, onde a seta que conecta a Ontologia de Perfil (no eProfile) à aplicação mostra o sentido do fluxo de dados.

4.2 Ontologias

O eProfile utiliza duas ontologias: uma ontologia de trilhas e uma ontologia de perfis. Para o armazenamento e processamento das ontologias, foram estudadas as linguagens de representação de conhecimento CycL[16], RDF/S[17], DAML+OIL[18] e OWL[18].

Devido à expressividade e ao suporte de ferramentas externas, o OWL foi escolhido como a linguagem de ontologias utilizado pelo eProfile para modelagem da ontologia do perfil e das trilhas.

As ontologias utilizadas pelo eProfile – tanto a ontologia de trilhas quanto a ontologia de perfil – são ontologias básicas, estendidas através de subclasses de acordo com o formato da trilha enviada pelo gerenciador (no caso da ontologia de trilha) ou das necessidades da aplicação (no caso da ontologia do perfil). Isto é representado na Figura 7, onde o eProfile oferece uma classe genérica "Atividade", que é especializada pela classe "AtividadeDeCompra", utilizada por um dos gerenciadores de trilha.

4.2.1 Ontologia de Trilha

O eProfile implementa uma ontologia de trilha, a qual será chamada **eOntoTrail**. Esta ontologia será alimentada a partir dos diferentes gerenciadores de trilhas suportados, onde o agente conversor é responsável por receber os dados no formato do gerenciador de trilhas e convertê-los para

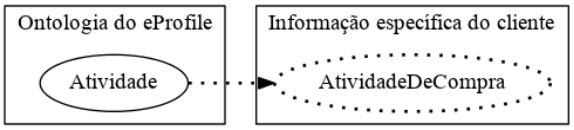

Figura 7: Exemplo da extensão de uma classe do eProfile

a eOntoTrail. Assim, o raciocinador terá uma ontologia única sobre a qual poderá raciocinar.

A eOntoTrail está representada na Figura 8, onde todos os relacionamentos, representados pelas setas, são do tipo ou "has-a" ou "has-some".

Abowd e Dey[19] definem que aplicações sensíveis a contexto estão interessadas em saber **quem**, **onde**, **quando** e **o quê** (isto é, o que o usuário está fazendo), de forma a poderem descobrir por que a situação está ocorrendo. Assim, uma trilha é representada por um conjunto de eventos caracterizados pelas seguintes classes:

Identity: É a identidade da entidade que está realizando a ação. Esta entidade pode estar realizando a ação com um ou mais recursos (*Resource*) (um aplicativo ou dispositivo, por exemplo) e pode estar vinculada a um perfil (*Profile*) dentro do eProfile. A classe **Participants** contém as identidades das entidades que estão participando deste evento;

Location: A localização onde está ocorrendo a ação. Uma localização pode ser uma localização física específica (como uma sala) ou não (uma estrada, por exemplo). A localização pode estar vinculada a uma região (*Region* que, adicionalmente, pode estar vinculada a outras subregiões e geralmente possui uma localização global (GPS) vinculada a ela. Outros tipos de localização podem ser extendidas a partir da classe;

Time: A data e hora em que ocorreu o evento;

Activity: Refere-se à ação que o usuário está executando (ou tentou executar). É a classe mais abrangente da ontologia, e a que possivelmente será a mais especializada pelos gerenciadores de trilha.

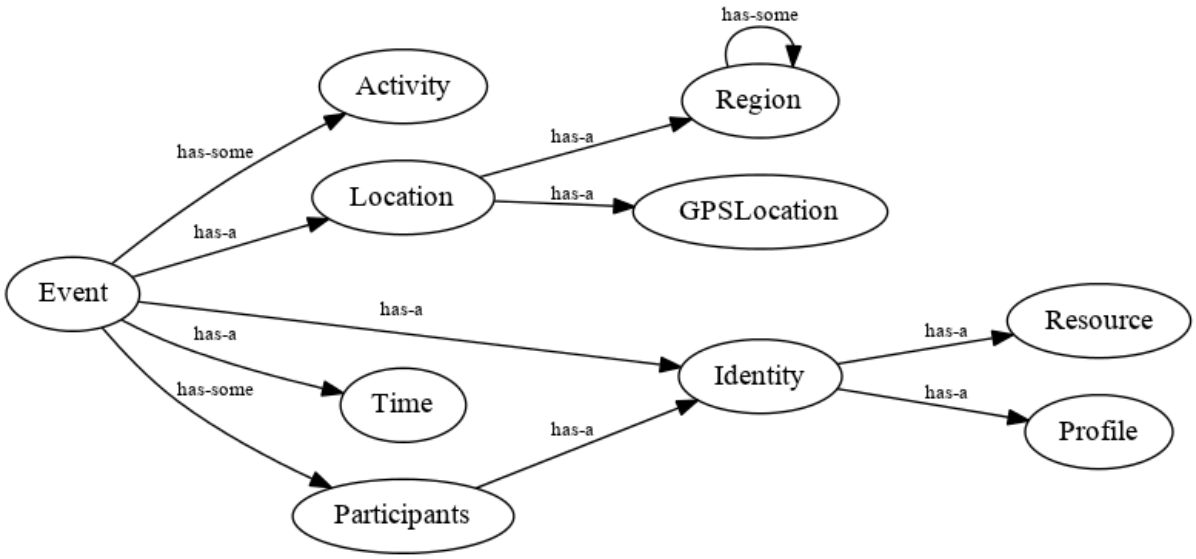

Figura 8: Ontologia de Trilha do eProfile

4.2.2 Ontologia de Perfil

A ontologia de perfil é a ontologia utilizada para armazenar os modelos de usuário, bem como as informações de perfil que são inferidas a partir da trilha. Para que a ontologia seja viável dentro do modelo proposto, ela deve ser uma ontologia genérica que permita que suas classes seja estendidas pelas aplicações.

Existem algumas ontologias genéricas propostas, tais como FOAF[20], GUMO[21], Ontonym[22], e a ontologia proposta por Golemati et al[23].

Na ontologia proposta, a qual chamaremos eOntoProfile, está representada na Figura ??. A eOntoProfile é uma ontologia básica, que será estendida pelas aplicações de modo a montarem seus próprios modelos de usuário.

A classe básica da ontologia é a classe *Element* (elemento). A classe *Element* representa qualquer coisa no mundo real. Ela tem como filhos as seguintes classes:

Entity: representa uma entidade. Uma entidade é qualquer elemento que tenha um perfil associado a ela. Embora a idéia de um perfil geralmente esteja associada a uma pessoa, uma entidade pode também ser um dispositivo como um carro ou um celular;

NonEntity: representa uma entidade que não possui um perfil associado a ela, como um objeto, um estabelecimento comercial, ou até mesmo uma pessoa que não tenha um perfil associado. Pode ser utilizada para identificar alguma relação que a entidade tenha com ela, por exemplo, um livro que uma pessoa possua ou uma loja que ela tenha visitado.

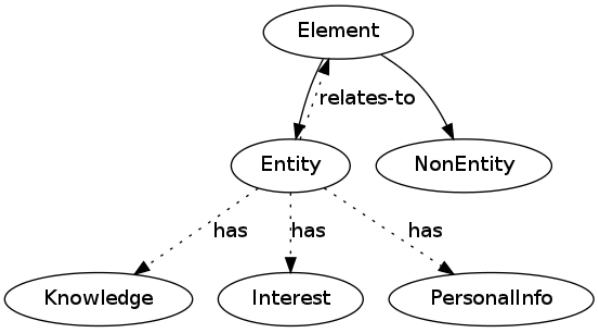

Figura 9: Ontologia de Perfil do eProfile

A ontologia FOAF[20] utiliza o conceito de *knows* (conhece) para vincular duas pessoas. O eOntoProfile estende este conceito através das propriedade *relates-to* (se-relaciona-com) que vincula dois elementos. Através desta propriedade, é possível não apenas vincular os elementos, mas também descrever o tipo de relação que eles tem.

Exemplos de relações são *friend-of* (amigo-de), representando uma amizade entre duas pessoas, *employee-of* (empregado-de), representando um vínculo empregatício entre duas pessoas, *owns* (possui), representando que uma pessoa possui um dispositivo, *depends* (depende), onde um dispositivo depende de outro, e qualquer outra relação que a aplicação deseje criar.

Uma entidade pode conter diversas características. As características das entidades são inspiradas no modelo proposto por Golemati et al[23], embora o modelo tenha sido simplificado.

PersonalInfo: contém quaisquer informações pessoais do usuário, tais como nome, data de nascimento, moradia, emprego atual, entre outros;

Interest: quaisquer interesses ou preferências que o usuário tenha, como preferência musical ou preferência por um tema de cores específico em suas aplicações;

Knowledge: conhecimento possuido pelo usuário. Pode referir-se a conhecimento formal (tal como formação acadêmica) ou informal (habilidade ou experiência em alguma atividade).

Os modelos de usuário estenderão estas categorias de acordo com as suas necessidades.

Todos os modelos de usuários definidos para um perfil serão estendidos em uma mesma ontologia. Isto possibilitará a interpersonalização, ou seja, permitirá com que os dados gerados por uma aplicação possam ser utilizadas por outra. Assim, existe a possibilidade de conflito de nomes, ou seja, uma aplicação pode criar uma classe chamada *Preference* onde ela registra preferências do usuário por um determinado tipo de livros, enquanto outro aplicativo cria uma classe com o mesmo nome para registrar as preferências de tela da aplicação.

Para evitar este conflito, um nome único de aplicação deve ser concatenado ao nome da classe. Assim, se um dos aplicativos utilizasse o nome *books* e o outro utilizasse o nome *gps*, teríamos, respectivamente, as classes *books.Preference* e *gps.Preference*.

Para que um aplicativo possa ter acesso aos dados gerados por outro, existem duas possibilidades:

1. Um aplicativo acessar diretamente os dados gerados pelo outro. No exemplo acima, o aplicativo *books* acessaria diretamente os dados da classe *gps.Preference*;

2. Os aplicativos utilizarem um contrato onde eles concordam em utilizar os mesmos campos. Assim, *books* e *gps* poderiam utilizar o nome *findlibrary*, criando o campo *findlibrary.Preference* onde ambos gravariam e leriam as informações.

Todos as extensões (subclasses) da ontologia são definidas pelas aplicações, e todas as informações do perfil são inferidas. A forma como esta inferência ocorre é descrita na Seção 4.4.

4.3 Agentes

Como o eProfile é um modelo onde é necessário inferir informações de forma autônoma a respeito de dados armazenados em uma ontologia, a tecnologia de agentes autônomos foi escolhida para o desenvolvimento.

O modelo do eProfile é composto por três agentes: (i) conversor, (ii) regulador e raciocinador.

O **agente conversor** é responsável por realizar a conversão dos dados da trilha do gerenciador de trilhas (externo) para a ontologia de trilha do eProfile (interno). A trilha do gerenciador pode ser disponibilizada em qualquer formato e por qualquer meio, desde que o eProfile ofereça suporte ao gerenciador de trilhas específico. O agente conversor tem também a responsabilidade de verificar, a cada período pré-determinado, atualizações nos dados disponibilizados no gerenciador.

O **agente regulador** realiza o recebimento das regras enviadas pela aplicação. Uma vez que as regras sejam recebidas, elas são validadas e, se forem consideradas válidas, são armazenadas na base de regras. A aplicação é então informada do aceite ou rejeição. O agente pode rejeitar uma regra se: (i) a regra for inválida ou (ii) se for verificado que a execução da regra irá consumir recursos excessivos do servidor.

O **agente raciocinador** é responsável pela inferência das trilhas de acordo com as regras enviadas pela aplicação, e posterior atualização do perfil da entidade. A execução do agente não necessita que o cliente ou o gerenciador de trilhas esteja conectado, e ocorre da seguinte forma:

1. O agente verifica as regras, comparando a data da útlima execução da regra com a freqüência de execução, definida pela aplicação na criação da regra.
2. Caso seja o momento da execução da regra, a mesma é executada através da biblioteca externa HermiT[24].
3. Caso a regra tenha sido executada com sucesso, o resultado da regra é atualizado no perfil. Em caso de insucesso, o erro é enviado para a aplicação no momento que a mesma se reconectar.

4.4 Regras e Inferência

Para que uma aplicação possa utilizar os serviços oferecidos pelo eProfile, é necessário que a aplicação envie ao eProfile a ontologia de perfil que ele irá utilizar, extendendo a ontologia do eProfile. A ontologia é enviada quando a aplicação faz seu primeiro acesso ao eProfile, e pode ser atualizada posteriormente. Ela deve ser enviada em formato OWL utilizando a sintaxe RDF/XML[15].

4.4.1 Regras de Inferência

A aplicação pode, a qualquer momento, criar, editar ou excluir regras para a inferência das informações de sua trilha, de modo a gerar informações úteis dentro da ontologia de perfil.

As regras devem ser enviadas em formato **SWRL**[25]. SWRL (*Semantic Web Rule Language*) é uma linguagem de regras proposta pela W3C. Esta linguagem foi escolhida pela expressividade que acrescenta à OWL.

Cada nova regra é gravada em uma base de regras. Assim, regras podem ser compartilhadas entre diferentes aplicações, desde que ambas extendam da mesma forma a ontologia de trilha.

Quando uma nova regra é enviada ao eProfile, ela deve conter as seguintes informações:

Nome: o nome da regra, para que possa ser atualizada ou excluída posteriormente;

Tipo de regra: se é uma regra em OWL, SPARQL ou SWRL;

Regra: a regra propriamente dita, na linguagem escolhida;

Objeto: classe, instância ou propriedade da ontologia de perfil que será atualizada;

Instrução de atualização: a forma como a instrução será atualizada;

Freqüência de atualização: a freqüência com que a regra deve ser atualizada;

Validade: a data limite até quando a regra será processada (pode ser ilimitada).

A instrução de atualização pode ser de um dos seguintes tipos:

Substituição: o item anterior é substituído por um novo item. Pode ser utilizado, por exemplo, em dados cadastrais como endereço e telefone, onde cada novo item substitui o anterior;

Adição: uma informação é adicionada, como a lista de endereços visitados por um carro;

Expressão: uma informação que precisa ser calculada. Neste caso, a própria aplicação passa a expressão do cálculo. Calcular o percentual de interesse de um usuário por um determinado assunto, por exemplo, seria uma informação deste tipo.

Uma vez que a regra tenha sido validada pelo agente regulador, ela entra em funcionamento e passa a ser elegível para execução.

As regras são executadas através de um motor de inferências externo. Foram estudados os seguintes raciocinadores que oferecem suporte a estas operações: Pellet[26], FaCT++[27] e HermiT[24]. Devido ao conjunto de funcionalidades e ao desempenho superior, o HermiT foi escolhido como o raciocinador utilizado pelo eProfile.

Um exemplo deste tipo de regra poderia ser a tentativa de identificação de parentesco entre duas entidades (neste caso, duas pessoas). Por exemplo, para identificar se duas pessoas são primas (isto é, se elas tem a propriedade `temPrimo` vinculando-as), a seguinte regra se aplicaria: `temPai(?a,?b)` $\wedge$ `temIrmao(?b,?c)` $\wedge$ `temFilho(?c,?d)` $\Rightarrow$ `temPrimo(?a,?d)`.

5 Validação

A validação deste modelo busca provar que o modelo apresentado é funcional e efetivo.

Para o teste de funcionalidade, o modelo será implementado e integrado com o eTrail, um gerenciador de trilhas. Uma vez que esta integração esteja realizada, um conjunto de regras será desenvolvido, de modo a verificar a capacidade de inferência do eProfile.

A seguir, será testada a capacidade do eProfile em gerenciar perfis de forma efetiva. Para tal, um grupo de usuários utilizará um aplicativo cujo perfil é gerenciado pelo eProfile. Após a utilização, será apresentado a estes usuários um questionário que buscará avaliar a efetividade do eProfile.

Referências

[1] M. Viviani, N. Bennani, and E. Egyed-Zsigmond, "A Survey on User Modeling in Multi-Application Environments," in *The Third International Conference on Advances in Human-oriented and Personalized Mechanisms, Technologies, and Services CENTRIC'10*, pp. 111–116, IEEE, Aug. 2010.

[2] A. Dey, "Understanding and using context," *Personal and Ubiquitous Computing*, vol. 6, no. 1, pp. 4–7, 2001.

[3] M. Weiser, "The computer for the 21st century," *Scientific America*, pp. 94–104, 1991.

[4] M. Weiser and J. S. Brown, "Designing calm technology," *PowerGrid Journal*, July 1996.

[5] J. Silva, J. Rosa, J. Barbosa, D. N. F. Barbosa, and L. A. M. Palazzo, "Content distribution in trial-aware environments," in *Proceedings of the XV Brazilian Symposium on Multimedia and the Web*, WebMedia '09, (New York, NY, USA), pp. 15:1–15:8, ACM, 2009.

[6] D. M. da Silva, J. Barbosa, and R. Vieira, "Octopus: um modelo para classificação de perfil de usuários usando trilhas em sistemas ubíquos," in *Companion Proceedings of the XIV Brazilian Symposium on Multimedia and the Web*, WebMedia '08, (New York, NY, USA), pp. 185–188, ACM, 2008.

[7] C. Niederée, A. Stewart, B. Mehta, and M. Hemmje, "A Multi-Dimensional, Unified User Model for Cross-System Personalization," in *E4PIA Workshop 2004*, 2004.

[8] IEEE, "IEEE standard computer dictionary. a compilation of IEEE standard computer glossaries," *IEEE Std 610*, p. 1, 1991.

[9] F. Carmagnola and F. Cena, "User identification for cross-system personalisation," *Inf. Sci.*, vol. 179, pp. 16–32, January 2009.

[10] D. Levis, J. L. V. Barbosa, and S. C. C. S. Pinto, "Aperfeiçoamento automático do perfil do aprendiz em ambientes de educação ubíqua," *Revista Brasileira de Informática na Educação*, vol. 16, no. 1, 2008.

[11] D. Barbosa, C. Geyer, and J. Barbosa, "Globaledu - an architecture to support learning in a pervasive computing environment," in *New Trends and Technologies in Computer-Aided Learning for Computer-Aided Design* (A. Rettberg and C. Bobda, eds.), vol. 192 of *IFIP International Federation for Information Processing*, pp. 1–10, Springer Boston, 2005.

[12] J. Barbosa, R. Hahn, S. Rabello, and D. Barbosa, "Local: a model geared towards ubiquitous learning," *SIGCSE Bull.*, vol. 40, pp. 432–436, March 2008.

[13] S. A. Chellouche, J. Arnaud, and D. Négru, "Flexible user profile management for context-aware ubiquitous environments," in *Proceedings of the 7th IEEE conference on Consumer communications and networking conference*, CCNC'10, (Piscataway, NJ, USA), pp. 980–984, IEEE Press, 2010.

[14] M. Benerecetti, P. Bouquet, and C. Ghidini, "Contextual reasoning distilled," *Philosophical Foundations of Artificial Intelligence. A special issue of the journal of Experimental and Theoretical AI (JETAI)*, vol. 12, no. 3, pp. 279–305, 2000.

[15] W3C, "RDF/XML syntax specification (revised)," 2011. `http://www.w3.org/RDF/`.

[16] D. B. Lenat and R. V. Guha, *Building Large Knowledge-Based Systems; Representation and Inference in the Cyc Project.* Boston, MA, USA: Addison-Wesley Longman Publishing Co., Inc., 1st ed., 1989.

[17] W3C, "RDF vocabulary description language 1.0: Rdf schema," 2011. `http://www.w3.org/TR/rdf-schema/`.

[18] M. C. Daconta, L. J. Obrst, and K. T. Smith, *The Semantic Web : A Guide to the Future of XML, Web Services, and Knowledge Management.* Indianapolis: Wiley, May 2003.

[19] G. D. Abowd, A. K. Dey, P. J. Brown, N. Davies, M. Smith, and P. Steggles, "Towards a better understanding of context and context-awareness," in

Proceedings of the 1st international symposium on Handheld and Ubiquitous Computing, HUC '99, (London, UK), pp. 304–307, Springer-Verlag, 1999.

[20] D. Brickley and L. Miller, "FOAF Vocabulary Specification 0.97," namespace document, January 2010.

[21] D. Heckmann, T. Schwartz, B. Brandherm, M. Schmitz, and M. von Wilamowitz-Moellendorff, "Gumo – The General User Model Ontology," pp. 428–432, 2005.

[22] G. Stevenson, S. Knox, S. Dobson, and P. Nixon, "Ontonym: a collection of upper ontologies for developing pervasive systems," in *Proceedings of the 1st Workshop on Context, Information and Ontologies*, CIAO '09, (New York, NY, USA), pp. 9:1–9:8, ACM, 2009.

[23] M. Golemati, A. Katifori, C. Vassilakis, G. Lepouras, and C. Halatsis, "Creating an ontology for the user profile: Method and applications," in *In Proceedings of the First International Conference on Research Challenges in Information Science (RCIS)*, 2007.

[24] B. Motik, R. Shearer, and I. Horrocks, "Hypertableau Reasoning for Description Logics," *Journal of Artificial Intelligence Research*, vol. 36, pp. 165–228, 2009.

[25] W3C, "SWRL: A semantic web rule language combining OWL and ruleml," 2004. `http://www.w3.org/Submission/SWRL/`.

[26] E. Sirin, B. Parsia, B. Grau, A. Kalyanpur, and Y. Katz, "Pellet: A practical OWL-DL reasoner," *Web Semantics: Science, Services and Agents on the World Wide Web*, vol. 5, pp. 51–53, June 2007.

[27] D. Tsarkov and I. Horrocks, "Fact++ description logic reasoner: System description," in *Proc. of the Int. Joint Conf. on Automated Reasoning (IJCAR 2006)*, vol. 4130 of *Lecture Notes in Artificial Intelligence*, pp. 292–297, Springer, 2006.